

LAPORAN PRATIKUM PEMROGRAMAN BERBASIS OBJEK

MODUL 1: OBJECT, CLASS, CONSTRUCTOR

Pekan 1

DI SUSUN OLEH :

ABDUL KARIM ALGAZALI

NIM 2511532029

DOSEN PENGAMPU : NURFIAH, S.ST, M.KOM

ASISTEN LABORATORIUM : Dawi Syauqi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa, karena atas rahmat dan izin-Nya laporan praktikum “Modul 1: Object, Class, Constructor” ini dapat diselesaikan dengan baik. Saya ucapkan terima kasih sebesar-besarnya kepada Bapak Nurfiah S.ST, M.Kom selaku dosen pengampu mata kuliah Pemrograman Berorientasi Objek serta kepada asisten laboratorium yang telah membimbing pelaksanaan praktikum ini.

Laporan ini disusun sebagai bentuk dokumentasi kegiatan praktikum Pekan 1 yang bertujuan untuk memahami konsep dasar pemrograman berorientasi objek, khususnya pembuatan class, object, constructor, atribut, method, serta interaksi pengguna melalui command line interface (CLI). Selain itu, praktikum ini juga menuntut penerapan struktur data ArrayList untuk menyimpan banyak objek dan pengelolaan state saldo pada studi kasus rekening bank.

Saya menyadari bahwa penulisan laporan praktikum ini masih jauh dari kata sempurna. Oleh karenanya, saya dengan rendah hati menerima saran, masukan, dan kritikan guna penyempurnaan laporan ini.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang	3
1.2 Tujuan	3
1.3 Manfaat	3
BAB II PEMBAHASAN.....	4
2.1 Implementasi Class Rekening.....	4
2.2 Implementasi Class Main.....	6
BAB III TUGAS.....	9
3.1 Implementasi Fitur Tarik Tunai	9
3.2 Implementasi Fitur Tarik Tunai	9
BAB IV KESIMPULAN.....	11
3.1 Kesimpulan	11
3.2 Saran	11
DAFTAR PUSTAKA.....	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam Pemrograman Berorientasi Objek (PBO), perangkat lunak dimodelkan berdasarkan entitas yang ada di dunia nyata. Dua pilar paling mendasar dalam pemodelan ini adalah Class dan Object. Class berfungsi sebagai cetak biru (blueprint) dari sebuah objek yang mendefinisikan struktur data (atribut atau state) serta sekumpulan operasi (metode atau behavior) yang dapat dilakukan pada data tersebut. Class itu sendiri tidak memakan ruang memori untuk menyimpan nilai data, melainkan hanya berfungsi sebagai tipe data referensi. Di sisi lain, Object adalah instansiasi konkret dari sebuah class yang dialokasikan di dalam memori komputer, di mana setiap objek memiliki state atau kondisi yang saling independen. Selain itu, PBO juga menggunakan Constructor, yaitu blok kode khusus yang dieksekusi secara otomatis pada saat objek dibuat untuk menginisialisasi state awal dari objek tersebut.

1.2 Tujuan

1. Mendefinisikan class sebagai model dari entitas dunia nyata.
2. Mendefinisikan object dan memahami alokasi memori independen.
3. Menggunakan constructor untuk inisialisasi awal state pada objek.
- 4.

1.3 Manfaat

1. Meningkatkan pemahaman mengenai manajemen memori dinamis di Java.
2. Meningkatkan fondasi logika untuk membangun struktur data dan arsitektur program yang lebih kompleks di masa depan.

BAB II

PEMBAHASAN

2.1 Implementasi Class Rekening

Code: package pekan1;

```
public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    public Rekening(String nomor, String nama, double saldoAwal)
    {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;
        System.out.println("Rekening atas nama" + namaPemilik
            + " berhasil dibuat dengan saldo Rp" + saldo);
    }

    public String getNomorRekening() {
        return nomorRekening;
    }

    public void setorTunai(double nominal) {
        if (nominal > 50000) {
            saldo += nominal;
            System.out.println("Saldo tunai Rp" + nominal
                + " berhasil, Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal : nominal setor harus lebih dari
50000");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Gagal : nominal tarik harus minimal
10000");
        } else if (saldo - nominal < 10000) {
            System.out.println("Gagal: Saldo tidak mencukupi (saldo
minimal tersisa Rp10.000).");
        } else {
            saldo -= nominal;
            System.out.println("Tarik tunai Rp" + nominal
                + " berhasil, Saldo saat ini: Rp" + saldo);
        }
    }
}
```

```

    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Terakhir: Rp" + saldo);
        System.out.println("-----");
    }
}

```

Pembuatan program perbankan menggunakan CLI agar praktikum dapat lebih berfokus pada logika memori. Pemodelan entitas diawali dengan membuat class Rekening yang bertindak sebagai model utama. Class ini memiliki tiga state atau atribut dasar, yaitu nomorRekening bertipe data String, namaPemilik bertipe data String, dan saldo bertipe data double.

Untuk menginisialisasi state awal dari objek tersebut, dibuat sebuah constructor Rekening(String nomor, String nama, double saldoAwal) yang menerima parameter nomor, nama, dan saldo awal. Constructor ini dieksekusi secara otomatis setiap kali objek Rekening baru dibuat.

Selain atribut, class ini juga dilengkapi dengan behavior berupa:

- `getNomorRekening()` — accessor (getter) yang mengembalikan nilai nomorRekening. Method ini ditambahkan agar atribut dapat diakses secara terkontrol, terutama saat pencarian akun di dalam ArrayList.
- `setorTunai(double nominal)` — memanipulasi state dengan menambahkan nilai saldo, dengan validasi bahwa nominal setor harus lebih dari Rp50.000.
- `tarikTunai(double nominal)` — memanipulasi state dengan mengurangi nilai saldo, dengan dua tahap validasi (nominal minimal Rp10.000 dan saldo tersisa minimal Rp10.000).
- `cekInformasi()` — menampilkan rincian data nomorRekening, namaPemilik, dan saldo ke layar.

2.2 Implementasi Class Main

Code : package pekan1;

```
import java.util.Scanner;
import java.util.ArrayList;
```

```
public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor tunai");
            System.out.println("3. Tarik tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("0. Keluar");
            System.out.print("Pilih Menu : ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukan nomor rekening: ");
                    String no = input.nextLine();

                    boolean sudahTerdaftar = false;
                    for (Rekening r : daftarRekening) {
                        if (r.getNomorRekening().equals(no)) {
                            sudahTerdaftar = true;
                            break;
                        }
                    }

                    if (sudahTerdaftar) {
                        System.out.println("Error: Nomor rekening sudah terdaftar!");
                    } else {
                        System.out.print("Masukan nama pemilik: ");
                        String nama = input.nextLine();
                    }
                }
            }
        }
    }
}
```

```

        System.out.print("Masukan saldo awal: ");
        double saldo = input.nextDouble();

        Rekening rekeningBaru = new Rekening(no, nama, saldo);
        daftarRekening.add(rekeningBaru);
        akunAktif = rekeningBaru;
        System.out.println("Rekening berhasil dibuat dan langsung aktif!");
    }
    break;

case 2:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf, anda belum memiliki nomor
rekening / belum login!");
    } else {
        System.out.print("Masukan nominal setor: ");
        double setor = input.nextDouble();
        akunAktif.setorTunai(setor);
    }
    break;

case 3:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf, anda belum memiliki nomor
rekening / belum login!");
    } else {
        System.out.print("Masukan nominal penarikan: ");
        double tarik = input.nextDouble();
        akunAktif.tarikTunai(tarik);
    }
    break;

case 4:
    if (akunAktif == null) {
        System.out.println("Error : anda belum membuka rekening / belum
login");
    } else {
        akunAktif.cekInformasi();
    }
    break;

case 5:
    if (daftarRekening.isEmpty()) {
        System.out.println("Belum ada rekening yang terdaftar. Silakan buka
rekening baru terlebih dahulu.");
    } else {
        System.out.print("Masukan nomor rekening yang ingin diaktifkan: ");
        String noTujuan = input.nextLine();
        boolean ditemukan = false;

```

```

        for (Rekening r : daftarRekening) {
            if (r.getNomorRekening().equals(noTujuan)) {
                akunAktif = r;
                ditemukan = true;
                System.out.println("Berhasil ganti akun!");
                break;
            }
        }

        if (!ditemukan) {
            System.out.println("Nomor rekening tidak ditemukan!");
        }
    }
    break;

case 0:
    isRunning = false;
    System.out.println("Sistem ditutup. Terima kasih!");
    break;

default:
    System.out.println("Pilihan tidak valid");
}
}

input.close();
}
}

```

Interaksi pengguna dalam manipulasi objek dilakukan melalui class Main. Pada kelas ini, program menggunakan fungsi Scanner untuk menerima input dari pengguna dan memanfaatkan struktur perulangan while-loop agar menu sistem perbankan mini dapat berjalan (bernilai true) hingga pengguna memilih opsi untuk keluar (bernilai false). Di dalam blok perulangan tersebut, program menggunakan struktur kendali switch-case untuk mengeksekusi perintah berdasarkan pilihan menu:

- **Menu 1 (Buka Rekening Baru)** — program melakukan instansiasi objek dengan memanggil constructor new Rekening(no, nama, saldo), lalu menambahkannya ke dalam ArrayList<Rekening> dan menetapkan sebagai akunAktif. Sebelum membuat objek, program melakukan pengecekan duplikasi nomor rekening menggunakan perulangan for-each dan method getNomorRekening() untuk mencegah nomor rekening yang sama terdaftar dua kali.
- **Menu 2 (Setor Tunai)** — memanggil behavior akunAktif.setorTunai(setor).
- **Menu 3 (Tarik Tunai)** — memanggil behavior akunAktif.tarikTunai(tarik).
- **Menu 4 (Cek Informasi)** — memanggil akunAktif.cekInformasi().
- **Menu 5 (Ganti Akun)** — melakukan pencarian objek Rekening di dalam ArrayList berdasarkan nomor rekening, lalu menetapkan sebagai akunAktif baru.

BAB III

TUGAS

3.1 Implementasi Fitur Tarik Tunai

Pada implementasi fitur tarik tunai, penambahan method tarikTunai(double nominal) dilakukan di dalam class Rekening. Logika pada method ini menggunakan struktur if-else if-else untuk melakukan dua tahap validasi:

1. Validasi pertama — memastikan bahwa parameter nominal yang diinputkan pengguna tidak kurang dari batas minimal penarikan, yaitu Rp10.000.
2. Validasi kedua — memastikan bahwa saldo setelah dikurangi nominal tidak tersisa kurang dari Rp10.000 (menjaga saldo minimal mengendap).

Apabila kedua validasi tersebut lolos, maka nilai saldo akan dikurangi sebesar nominal dan program mencetak status bahwa transaksi berhasil.

```
Code: public void tarikTunai(double nominal) {  
    if (nominal < 10000) {  
        System.out.println("Gagal : nominal tarik harus minimal 10000");  
    } else if (saldo - nominal < 10000) {  
        System.out.println("Gagal: Saldo tidak mencukupi (saldo minimal tersisa  
Rp10.000).");  
    } else {  
        saldo -= nominal;  
        System.out.println("Tarik tunai Rp" + nominal  
            + " berhasil, Saldo saat ini: Rp" + saldo);  
    }  
}
```

Pada class Main, pemanggilan method ini diimplementasikan di dalam blok case 3. Sebelum method utama dipanggil, sistem melakukan pengkondisian if (akunAktif == null) untuk memastikan bahwa pengguna telah membuka atau memilih objek rekening yang valid di dalam memori. Jika kondisi tersebut terpenuhi, program meminta input nominal penarikan melalui Scanner.nextDouble() dan mengeksekusi akunAktif.tarikTunai(tarik).

3.2 Implementasi Fitur Multi Akun

Pada awal praktikum, aplikasi Main hanya dapat menyimpan satu akun (Rekening akunAktif = null;). Jika pengguna memilih menu "Buka Rekening Baru" untuk kedua kalinya, akun lama akan tertimpa. Untuk mengatasi keterbatasan tersebut, arsitektur penyimpanan diubah menggunakan struktur data Collection, yaitu ArrayList<Rekening>:

```
ArrayList<Rekening> daftarRekening = new ArrayList<>();  
Rekening akunAktif = null;
```

Setiap kali pengguna membuka rekening baru, objek Rekening yang terbentuk ditambahkan ke dalam ArrayList melalui daftarRekening.add(rekeningBaru).

Dengan demikian, seluruh objek rekening tetap tersimpan di dalam memori dan dapat diakses kembali kapan saja.

Sebagai bonus, ditambahkan menu baru yaitu "**Ganti Akun**" (menu nomor 5). Sistem akan meminta nomor rekening, kemudian mencari objek yang cocok di dalam ArrayList menggunakan perulangan for-each dan method `getNomorRekening()`:

Code : case 5:

```
if (daftarRekening.isEmpty()) {
    System.out.println("Belum ada rekening yang terdaftar. "
        + "Silakan buka rekening baru terlebih dahulu.");
} else {
    System.out.print("Masukan nomor rekening yang ingin diaktifkan: ");
    String noTujuan = input.nextLine();
    boolean ditemukan = false;

    for (Rekening r : daftarRekening) {
        if (r.getNomorRekening().equals(noTujuan)) {
            akunAktif = r;
            ditemukan = true;
            System.out.println("Berhasil ganti akun!");
            break;
        }
    }

    if (!ditemukan) {
        System.out.println("Nomor rekening tidak ditemukan!");
    }
}
break;
```

Dengan mekanisme ini, pengguna dapat memiliki banyak rekening dalam satu sesi program, dan setiap transaksi (setor/tarik/cek) akan selalu mengarah ke objek `akunAktif` yang sedang dipilih.

BAB IV KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Class dan Object merupakan pilar utama dalam memodelkan perangkat lunak berbasis Pemrograman Berorientasi Objek (PBO). Class berfungsi sebagai cetak biru (blueprint) dari entitas dunia nyata, sedangkan Object adalah bentuk instansiasi konkret yang dialokasikan di dalam memori komputer dan diinisialisasi nilai awalnya melalui eksekusi Constructor.

Penggunaan pustaka Collection pada Java, khususnya ArrayList, terbukti penting untuk mengatasi keterbatasan penyimpanan variabel tunggal. Arsitektur data dinamis ini memungkinkan program untuk menyimpan, menelusuri, dan mengelola banyak instansiasi objek Rekening sekaligus dalam satu waktu. Hal ini mendukung terbentuknya fungsionalitas sistem perbankan multi-akun yang lebih kompleks, fleksibel, dan terstruktur.

Selain itu, penambahan validasi seperti pengecekan duplikasi nomor rekening, batas minimal nominal setor, batas minimal penarikan, dan saldo minimal mengendap menunjukkan bahwa method (behavior) pada sebuah class berperan penting dalam menjaga integritas state objek, sehingga program tidak mudah mengalami kesalahan (error) saat dijalankan.

3.2 Saran

1. Penggunaan visualisasi alokasi memori (heap) dan referensi objek melalui diagram atau animasi sederhana akan sangat membantu mahasiswa dalam memahami perbedaan antara *class* sebagai *blueprint* dan *object* sebagai *instance*, serta bagaimana ArrayList<Rekening> menyimpan banyak objek dan bagaimana akunAktif merujuk pada salah satu objek tersebut. Visualisasi ini juga dapat memperjelas konsep *pass-by-reference* saat objek dimasukkan ke dalam collection.
2. Penambahan materi pra-praktikum melalui iLearn mengenai struktur data Collection (khususnya ArrayList) dan pengelolaan *exception* (seperti InputMismatchException pada Scanner) tetap perlu dilanjutkan agar mahasiswa memiliki fondasi konseptual yang matang sebelum mengimplementasikan sistem multi-akun dan validasi input yang lebih kompleks di laboratorium. Materi ini dapat disertai contoh kasus sederhana agar mahasiswa lebih siap menghadapi tantangan saat praktikum.

DAFTAR PUSTAKA

- [1] Nurfiah. *Modul Praktikum PBO - Java: Praktikum 1 Class, Object, Constructor dan Method*. [Diakses: 10 September 2026].